

Technologies Final Project:

Prediction Market Protocol

Made by: Yernar Katyshev, Nursalim Onalbayev,
Zhalgas Torekeldi

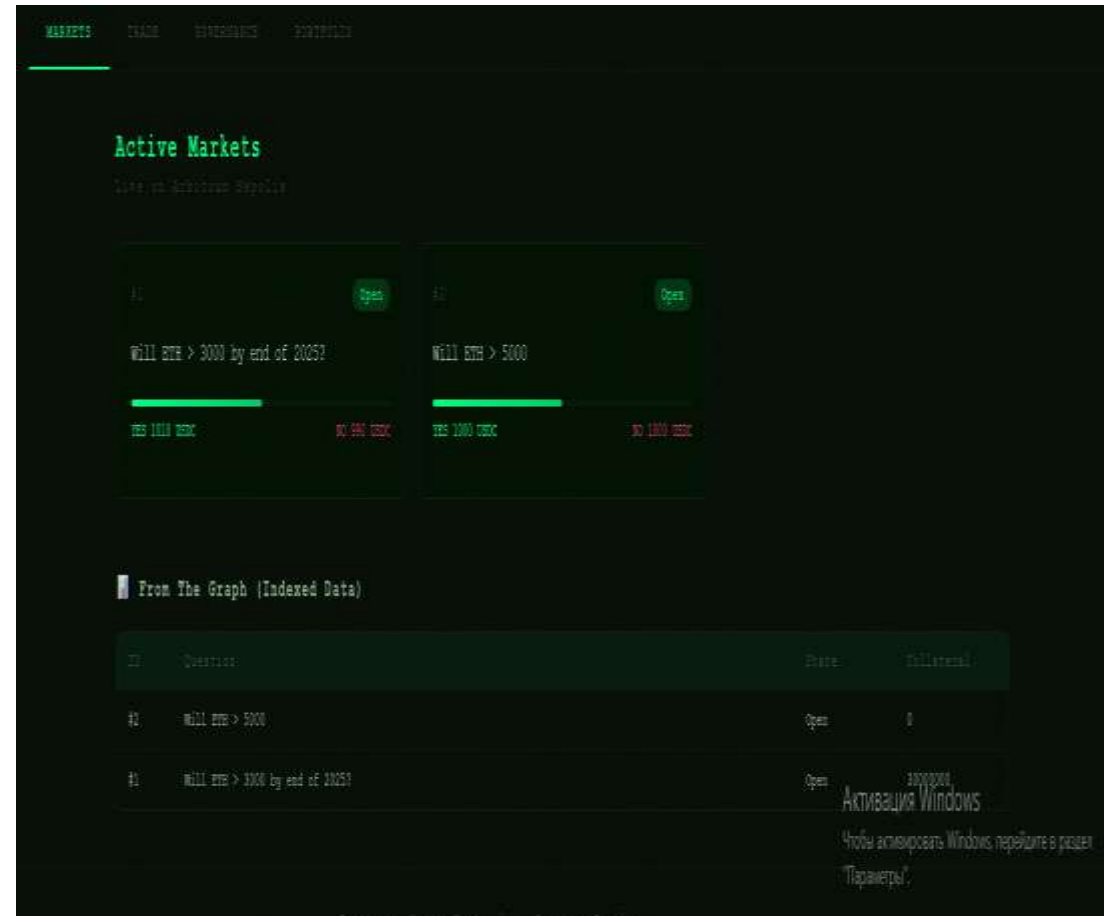
Group: SE-2425

Prediction markets need trustless trading, transparent settlement, and automated resolution.

- Traditional prediction markets are:
 - centralized
 - non-transparent
 - difficult to audit
 - dependent on trusted intermediaries
- We developed a decentralized prediction market protocol where users can:
 - create binary prediction markets
 - buy YES/NO shares
 - provide liquidity
 - participate in governance

Main Features

- Binary outcome markets
- CPMM AMM pricing mechanism
- ERC-1155 outcome tokens
- ERC-4626 yield vault
- Chainlink oracle integration
- DAO governance
- L2 deployment on Arbitrum Sepolia
- Upgradeable contracts (UUPS)



System Architecture

- **Main Components**

- PredictionMarket (main protocol logic)
- OutcomeToken (ERC-1155 YES/NO shares)
- GovernanceToken (ERC20Votes)
- FeeVault (ERC-4626 vault)
- OracleAdapter (Chainlink integration)
- PredictionGovernor
- MarketTimelock
- MarketFactory

- **Workflow**

- User → Frontend → Smart Contracts → Oracle → Subgraph → Frontend

- **Mention**

- Frontend built with React + Ethers.js/Viem.

Smart Contract Architecture

- **PredictionMarket**

- Responsible for:
- market creation
- liquidity management
- trading outcome shares
- market resolution
- reward redemption

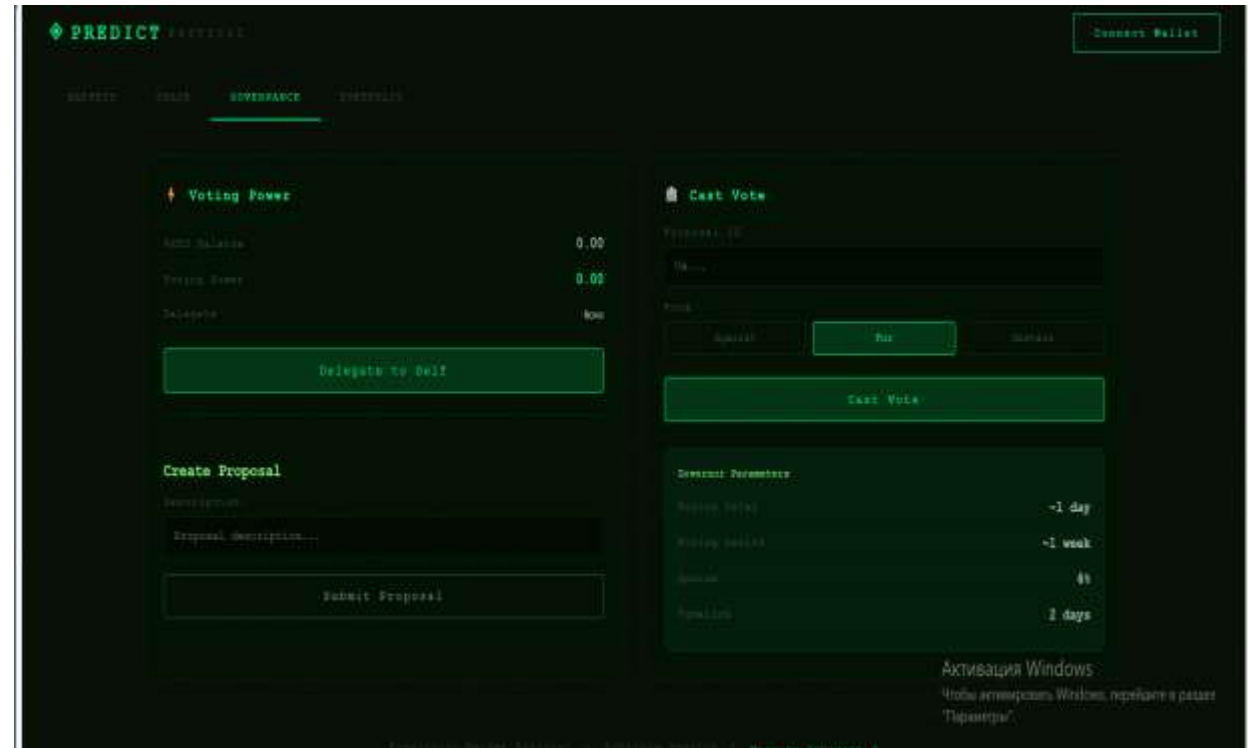
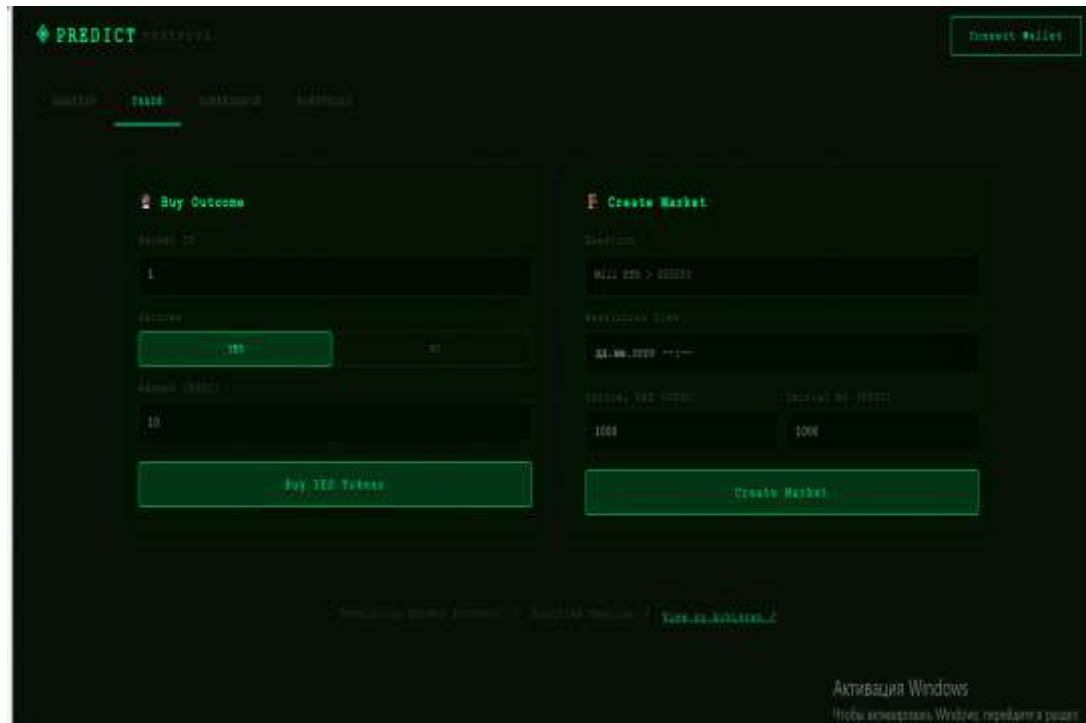
- **OutcomeToken (ERC-1155)**

- YES and NO shares
- tokenId structure:
 - $\text{YES} = \text{marketId} * 2$
 - $\text{NO} = \text{marketId} * 2 + 1$

- **GovernanceToken**

- ERC20Votes
- ERC20Permit support
- delegation support
- voting power tracking

DAPp frontend



Dapp's frontend

```
// — Contract addresses (Arbitrum Sepolia) —
const ADDRESSES = {
  market: "0x8bE287Cddb210165a7F47Fc04a04Ccc26E9a335A",
  govToken: "0x2d041dec8f01f50741B3808721be1077680e15A8",
  usdc: "0xb29e8cd93058d65ecD78A7538D23a9B7C3F9a74",
  outcomeToken: "0x286bB5A85Baa67A17A7F5379d09C0562425DF462",
  governor: "0x6d221157AE69fA5e7516fcfd513d3e37cE335cAe",
  feeVault: "0x2213868317b468b0c058D3EF70c078888eC6e7D8",
  oracle: "0x46F89e2315f0095b87A190E06774a042372aB23C",
  timelock: "0x2482f087466d97D0a70b87153888AD6760e417f2",
  factory: "0xeAd8FD78471c703cfbc645D3c9bc5Cf41C6E6b5",
};

const SUBGRAPH_URL = "https://api.studio.thegraph.com/query/1753417/final/v0.0.1";

const ARBITRUM_SEPOLIA_CHAIN_ID = 421614;

// — Minimal ABIs —
const MARKET_ABI = [
  "function createMarket(string,address,uint256,uint256,uint256,uint256) returns (uint256)",
  "function buyOutcome(uint256,bool,uint256,uint256) external",
  "function addLiquidity(uint256,uint256,uint256) external",
  "function removeLiquidity(uint256,uint256) external",
  "function resolveMarket(uint256) external",
  "function finalizeResolution(uint256) external",
  "function redeem(uint256) external",
  "function getMarketState(uint256) view returns (uint8)",
  "function getMarketOutcome(uint256) view returns (uint8)",
  "function getMarketQuestion(uint256) view returns (string)",
  "function getPoolReserves(uint256) view returns (uint256,uint256)",
  "function marketCount() view returns (uint256)",
  "function lpShares(address,uint256) view returns (uint256)",
];

const ERC20_ABI = [
  "function balanceOf(address) view returns (uint256)",
  "function approve(address,uint256) external",
  "function decimals() view returns (uint8)",
```

```
const GOVERNOR_ABI = [
  "function castVote(uint256,uint8) external",
  "function queue(address[],uint256[],bytes[],bytes32) external",
  "function execute(address[],uint256[],bytes[],bytes32) external",
  "function state(uint256) view returns (uint8)",
  "function proposalVotes(uint256) view returns (uint256,uint256,uint256)",
];

const STATE_LABELS = ["Open", "PendingResolution", "Resolved", "Disputed", "Cancelled"];
const OUTCOME_LABELS = ["Unresolved", "YES", "NO"];
const PROPOSAL_STATES = ["Pending", "Active", "Canceled", "Defeated", "Succeeded", "Queued", "Expired", "Executed"];

// — Subgraph query —
async function fetchMarketsFromSubgraph() {
  try {
    const res = await fetch(SUBGRAPH_URL, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        query: `
          {
            markets(orderBy: createdAt, orderDirection: desc, first: 10) {
              id question state outcome totalCollateral createdAt
            }
          }
        `
      })
    });
    const json = await res.json();
    return json?.data?.markets || [];
  } catch { return []; }
}

// — Main App —
export default function App() {
  const [provider, setProvider] = useState(null);
  const [signer, setSigner] = useState(null);
  const [account, setAccount] = useState(null);
  const [chainId, setChainId] = useState(null);
  const [tab, setTab] = useState("markets");
  const [markets, setMarkets] = useState([]);
```

```
// Use mock oracle feed address
const MOCK_FEED = "0x0000000000000000000000000000000000000000000000000000000000000000";
const restime = Math.floor(new Date(createInitTime).getTime() / 1000);
const feeData = await provider.getFeeData();
const tx = await mc.createMarket({
  createQuestion: MOCK_FEED, 30000000000n, restime,
  ethers.parseUnits(createInitYes, 6),
  ethers.parseUnits(createInitNo, 6),
  {
    maxFeePerGas: feeData.maxFeePerGas * 2n,
    maxPriorityFeePerGas: feeData.maxPriorityFeePerGas * 2n,
  }
});

await tx.wait();
setTxStatus("🟢 Market created! TX: " + tx.hash.slice(0,10) + "...");
await loadMarkets(provider);
} catch (e) { setTxStatus("❌ " + (e.reason || e.message)); }
setLoading(false);
}

async function handleDelegate() {
  if (!signer) { setTxStatus("Connect wallet first"); return; }
  setLoading(true); setTxStatus("Delegating votes...");
  try {
    const gov = new ethers.Contract(ADDRESSES.govToken, ERC20_ABI, signer);
    const feeData = await provider.getFeeData();
    const tx = await gov.delegate(account, {
      maxFeePerGas: feeData.maxFeePerGas * 2n,
      maxPriorityFeePerGas: feeData.maxPriorityFeePerGas * 2n,
    });
    await tx.wait();
    setTxStatus("Delegated to self!");
    await loadBalances(provider, signer, account);
  } catch (e) { setTxStatus("❌ " + (e.reason || e.message)); }
  setLoading(false);
}
```

```
// --- Transactions ---
async function approveUSD(amount) {
  const usdc = new ethers.Contract(ADDRESSES.usdc, ERC20_ABI, signer);
  const feeData = await provider.getFeeData();
  const tx = await usdc.approve(ADDRESSES.market, ethers.parseUnits(amount, 6), {
    maxFeePerGas: feeData.maxFeePerGas * 2n,
    maxPriorityFeePerGas: feeData.maxPriorityFeePerGas * 2n,
  });
  await tx.wait();
}

async function handleBuy() {
  if (!signer) { setTxStatus("Connect wallet first"); return; }
  setLoading(true); setTxStatus("Approving USDC...");
  try {
    await approveUSD(buyAmount);
    setTxStatus("🟢 Buying outcome...");
    const mc = new ethers.Contract(ADDRESSES.market, MARKET_ABI, signer);
    const feeData = await provider.getFeeData();
    const tx = await mc.buyOutcome(
      buyMarketId, buyYes,
      ethers.parseUnits(buyAmount, 6), {
        maxFeePerGas: feeData.maxFeePerGas * 2n,
        maxPriorityFeePerGas: feeData.maxPriorityFeePerGas * 2n,
      }
    );
    await tx.wait();
    setTxStatus("🟢 Bought ${buyYes}usd ? 'YES' : 'NO' token! TX: " + tx.hash.slice(0,10) + "...");
    await loadMarkets(provider);
    await loadBalances(provider, signer, account);
  } catch (e) { setTxStatus("❌ " + (e.reason || e.message)); }
  setLoading(false);
}

async function handleCreateMarket() {
```

```
// --- Connect wallet ---
async function connectWallet() {
  if (!window.ethereum) { setTxStatus("❌ MetaMask not found"); return; }
  try {
    const prov = new ethers.BrowserProvider(window.ethereum);
    const network = await prov.getNetwork();
    setChainId(Number(network.chainId));

    if (Number(network.chainId) !== ARBITRUM_SEPOLIA_CHAIN_ID) {
      setTxStatus("⚠️ Wrong network! Switching to Arbitrum Sepolia...");
      try {
        await window.ethereum.request({
          method: "wallet_switchEthereumChain",
          params: [{ chainId: "0x660000" }],
        });
      } catch {
        await window.ethereum.request({
          method: "wallet_addEthereumChain",
          params: [
            {
              chainId: "0x660000",
              chainName: "Arbitrum Sepolia",
              rpcUrls: ["https://sepolia-rollup.arbitrum.io/rpc"],
              nativeCurrency: { name: "ETH", symbol: "ETH", decimals: 18 },
              blockExplorerUrl: ["https://sepolia.arbiscan.io"],
            },
          ],
        });
      }
    }
    return;
  }

  const sig = await prov.getSigner();
  const addr = await sig.getAddress();
  setProvider(prov); setSigner(sig); setAccount(addr);
  setTxStatus("🟢 Connected: " + addr.slice(0,6) + "... " + addr.slice(-4));
  await loadBalances(prov, sig, addr);
  await loadMarkets(prov);
}
```

Dapp's frontend

Deploy status test and graphsql code.

Suite	Tests	Status
Unit (PredictionMarket)	37	Done
Unit (Tokens + Factory)	41	Done
Unit (Governance)	22	Done
Fuzz	10	Done
Invariant	5	Done
Fork	3	Done
Total	**118**	**Done**

```
1 question: String!
2 oracleEnd: BigInt!
3 resolutionTime: BigInt!
4 resolutionTime: BigInt!
5 state: String!
6 outcome: String!
7 createdAt: BigInt!
8 totalCollateral: BigInt!
9
10 type Trade @entity {
11   id: ID!
12   market: Market!
13   buyer: Bytes!
14   seller: Boolean!
15   amount: BigInt!
16   amountOut: BigInt!
17   timestamp: BigInt!
18   blockNumber: BigInt!
19 }
```

```
1 address alice = address("alice");
2 address bob = address("bob");
3
4 function setup() public {
5   vm.createSelector(vm.contract("MARKET_PROXY"));
6
7   vm.startPrank(alice);
8   OutcomeToken = new OutcomeToken(address(bob));
9   factory = new Factory(factory, address(bob));
10   oracle = new OracleAdapter();
11
12   PredictionMarketImpl = new PredictionMarket();
13   IMarketProxy proxy = new IMarketProxy(
14     address(factory),
15     address(alice),
16     address(bob),
17     address(factory),
18     address(bob)
19   );
20   IMarketProxy proxy = new IMarketProxy(
21     address(factory),
22     address(bob),
23     address(bob),
24     address(bob),
25     address(bob)
26   );
27 }
```

Gas report (for each unit)

Unit results: 0 OK, 0 passed, 0 failed, 0 skipped, finished in 120.92s (481.00s gas time)

lib/openzeppelin-contracts/contracts/proxy/ERC1967/ERC1967Proxy.sol:ERC1967Proxy Contract					
Deployment Cost	Deployment Size				
368883	1256				
Function Name	Min	Avg	Median	Max	# Calls
fallback	785	93837	117211	478100	161699

src/core/PredictionMarket.sol:PredictionMarket Contract					
Deployment Cost	Deployment Size				
2766219	12613				
Function Name	Min	Avg	Median	Max	# Calls
addLiquidity	27009	98831	97393	131593	33551
buyOutcome	24908	113758	113446	160746	66167
createMarket	3193	308841	436884	458230	51
disputeResolution	3857	9438	9438	13884	2
finalizeResolution	6911	9525	9899	9899	8

src/factories/MarketFactory.sol:MarketFactory Contract					
Deployment Cost	Deployment Size				
900504	4061				
Function Name	Min	Avg	Median	Max	# Calls
deployMarket	25828	306085	396654	485284	4
deployMarketDeterministic	28647	330967	431741	431741	4
deployedMarkets	4667	4667	4667	4667	1
getDeployedMarkets	7288	7288	7288	7288	1
implementation	2414	2414	2414	2414	1
predictAddress	7970	7970	7970	7970	1
saltToMarket	2508	2508	2508	2508	1
setImplementation	23804	26428	26428	29052	2

Gas report

src/governance/MarketTimelock.sol:MarketTimelock Contract					
Deployment Cost	Deployment Size				
1630181	7723				
Function Name	Min	Avg	Median	Max	# Calls
CANCELLER_ROLE	283	283	283	283	35
PROPOSER_ROLE	284	284	284	284	35
getMinDelay	2401	2401	2401	2401	3
grantRole	51508	51508	51508	51508	68
hasRole	2693	2693	2693	2693	2
hashOperationBatch	2451	2451	2451	2451	2
isOperationPending	2691	2703	2691	2729	3

src/governance/PredictionGovernor.sol:PredictionGovernor Contract					
Deployment Cost	Deployment Size				
3952748	19872				
Function Name	Min	Avg	Median	Max	# Calls
cancel	48984	48984	48984	48984	1
castVote	33755	76617	88646	88700	14
execute	79490	83979	83979	88468	2
getVotes	14246	14246	14246	14246	2
name	3347	3347	3347	3347	1
proposalNeedsQueuing	409	409	409	409	1
proposalThreshold	2483	2483	2483	2483	2
proposalVotes	6846	6846	6846	6846	3
propose	41133	78844	80378	88378	17
queue	38363	112163	149063	149063	1
quorum	18732	18732	18732	18732	1
quorumNumerator	4738	4738	4738	4738	2
state	2905	19348	11845	42262	1
timelock	2461	2461	2461	2461	1

Gas report

src/oracles/MockAggregator.sol:MockAggregator Contract						
Deployment Cost	Deployment Size					
240640	756					
Function Name	Min	Avg	Median	Max	# Calls	
decimals	2300	2300	2300	2300	1	
description	462	462	462	462	1	
getRoundData	4666	4666	4666	4666	1	
latestRoundData	4573	4573	4573	4573	14	
setAnswer	28676	31242	31476	31476	12	
setUpdatedAt	26439	26439	26439	26439	1	
version	221	221	221	221	1	

src/oracles/OracleAdapter.sol:OracleAdapter Contract						
Deployment Cost	Deployment Size					
224308	820					
Function Name	Min	Avg	Median	Max	# Calls	
getPrice	19229	19229	19229	19229	2	
requireFreshPrice	8286	9141	8286	19355	13	

src/tokens/FeeVault.sol:FeeVault Contract						
Deployment Cost	Deployment Size					
1242430	6218					
Function Name	Min	Avg	Median	Max	# Calls	
balanceOf	2637	2637	2637	2637	2	
deposit	51165	89098	108065	108065	3	
depositFee	23740	47952	60059	60059	3	
previewDeposit	8839	8839	8839	8839	1	
redeem	44379	44379	44379	44379	1	

Gas report

src/tokens/GovernanceToken.sol:GovernanceToken Contract						
Deployment Cost		Deployment Size				
2426469		11112				
Function Name	Min	Avg	Median	Max	# Calls	
DOMAIN_SEPARATOR	6347	6347	6347	6347	1	
allowance	2801	2801	2801	2801	1	
balanceOf	2649	2649	2649	2649	1	
clock	401	401	401	401	100	
delegate	27357	74153	74603	74603	105	
getPastTotalSupply	5624	5624	5624	5624	10	
getPastVotes	3215	5776	5859	5859	32	
getVotes	5024	5024	5024	5024	2	
initialize	237310	237310	237310	237310	45	
mint	2625	4131	4885	4885	3	
name	3310	3310	3310	3310	1	
nonces	2750	2750	2750	2750	1	
owner	2375	2375	2375	2375	1	
permit	57473	57473	57473	57473	1	

src/tokens/MockUSDC.sol:MockUSDC Contract						
Deployment Cost		Deployment Size				
665999		3405				
Function Name	Min	Avg	Median	Max	# Calls	
balanceOf	2560	2560	2560	2560	2	
decimals	2400	2400	2400	2400	1	
mint	24253	38904	38904	53555	2	
name	3174	3174	3174	3174	1	
symbol	3195	3195	3195	3195	1	

src/tokens/OutcomeToken.sol:OutcomeToken Contract						
Deployment Cost		Deployment Size				
1673515		7666				
Function Name	Min	Avg	Median	Max	# Calls	
MINTER_ROLE	237	237	237	237	64	
balanceOf	2657	2657	2657	2657	272	
burn	24684	29852	29852	35021	2	

Gas report

test/fuzz/FuzzInvariant.sol:MarketHandler Contract					
Deployment Cost	Deployment Size				
1592379	7449				
Function Name	Min	Avg	Median	Max	# Calls
addLiq	136744	138504	137499	191611	32039
buyNo	141511	146060	144021	249423	32278
buyYes	141363	145882	143863	249265	32027
removeLiq	15084	126670	131433	132154	32156

test/fuzz/FuzzInvariant.sol:MockERC20 Contract					
Deployment Cost	Deployment Size				
512750	2354				
Function Name	Min	Avg	Median	Max	# Calls
approve	46679	46679	46679	46679	25
balanceOf	2559	2559	2559	2559	514
decimals	265	265	265	265	15

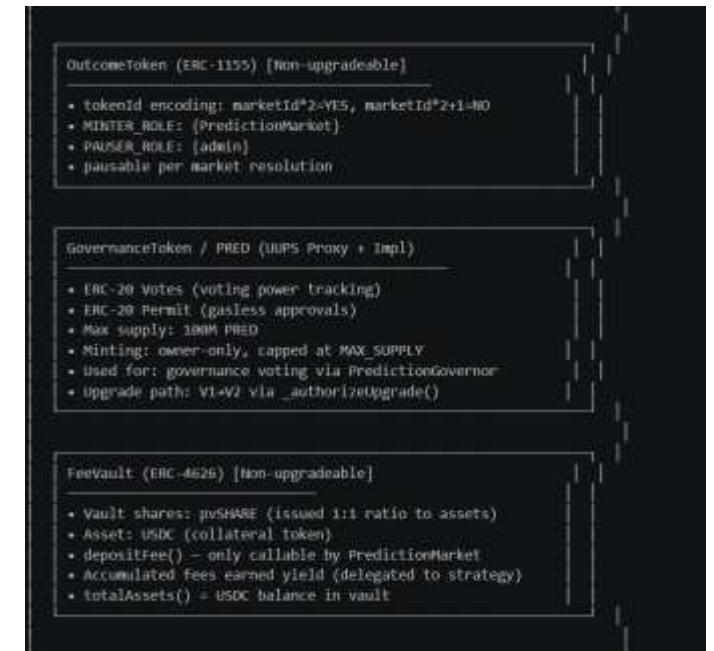
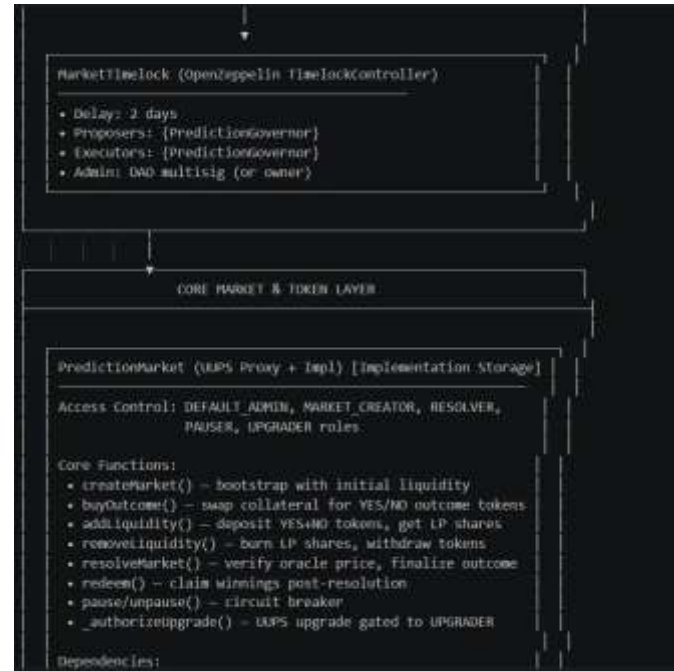
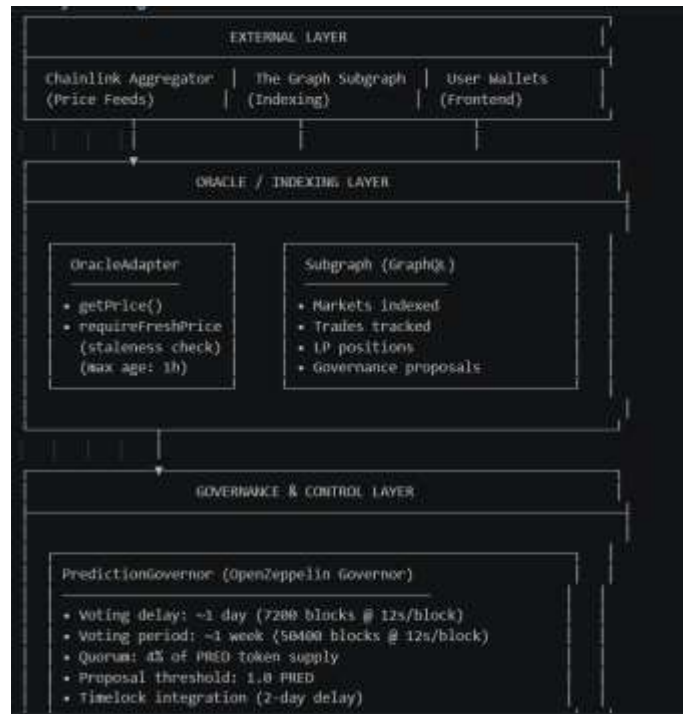
test/unit/TokenTest.sol:MockERC20 Contract					
Deployment Cost	Deployment Size				
512750	2354				
Function Name	Min	Avg	Median	Max	# Calls
approve	46679	46679	46679	46679	14
balanceOf	2559	2559	2559	2559	9
decimals	265	265	265	265	15
mint	51292	59842	59842	68392	14

test/unit/Unt.sol:MockERC20 Contract					
Deployment Cost	Deployment Size				
512750	2354				
Function Name	Min	Avg	Median	Max	# Calls
approve	46679	46679	46679	46679	111
balanceOf	2559	2559	2559	2559	10
decimals	265	265	265	265	37

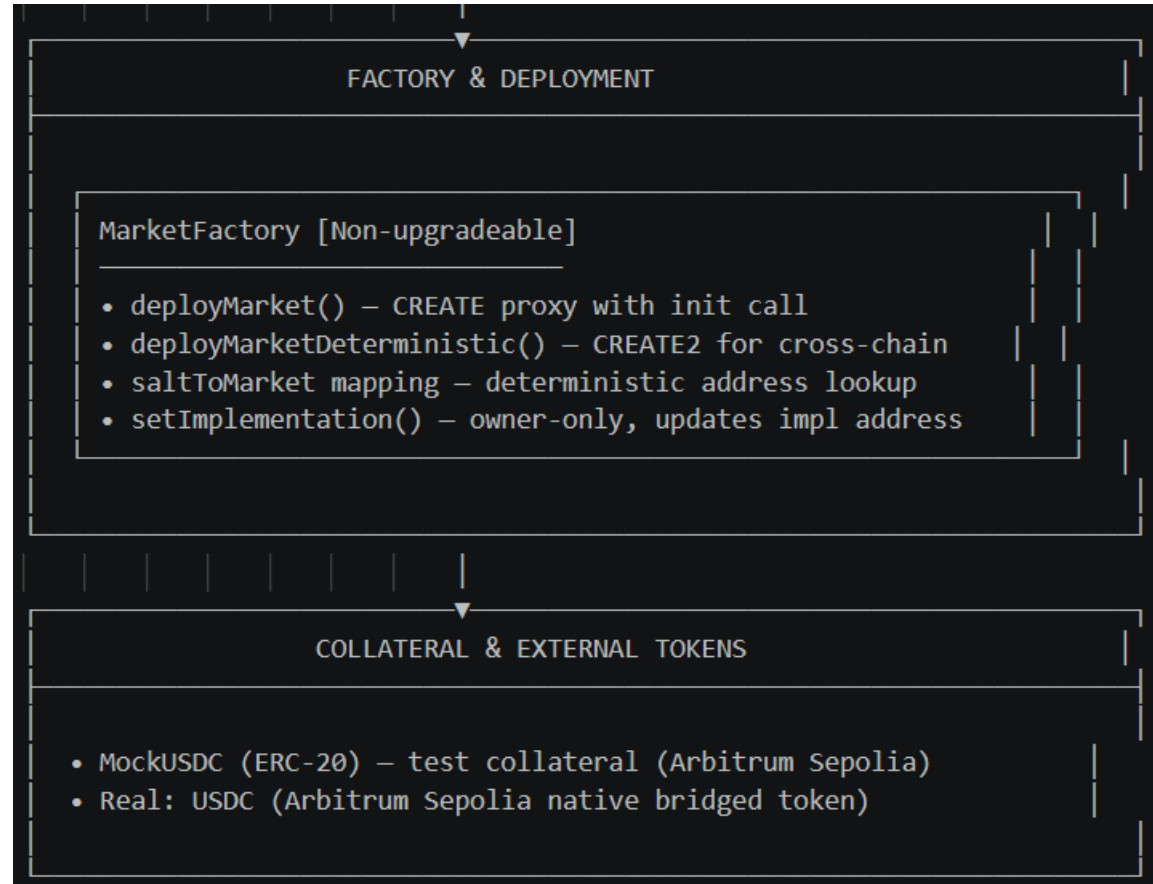
Architecture

- – ****CPMM AMM****: $x*y=k$ with 0.3% fee, Yul-optimized swap calculation
- – ****ERC-1155 Outcome Tokens****: YES/NO shares per market ($\text{tokenId} = \text{marketId} * 2 / \text{marketId} * 2 + 1$)
- – ****ERC-4626 Fee Vault****: LP fee accumulation with yield sharing
- – ****Chainlink Oracle****: Price feed with staleness check (1 hour max age)
- – ****UUPS Upgradeable****: PredictionMarket and GovernanceToken with V1→V2 upgrade path
- – ****DAO Governance****: Governor + TimelockController (2-day delay), 4% quorum, 1-week voting
-

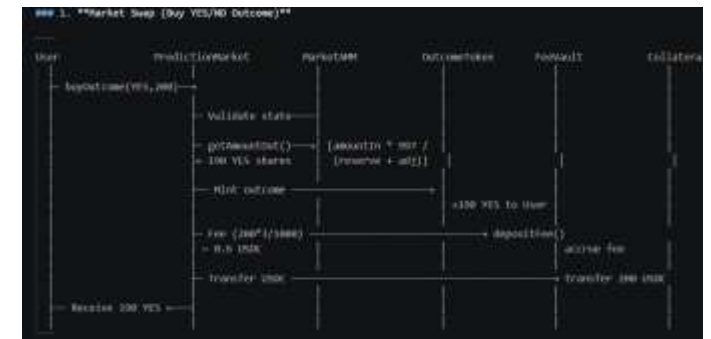
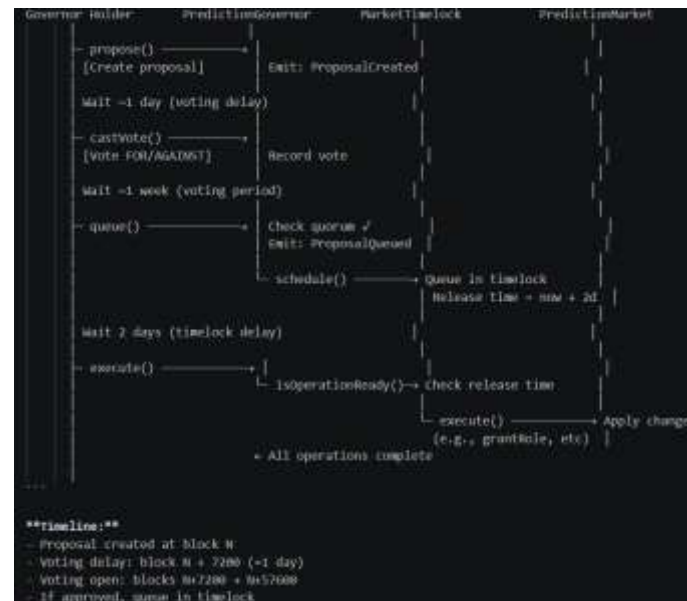
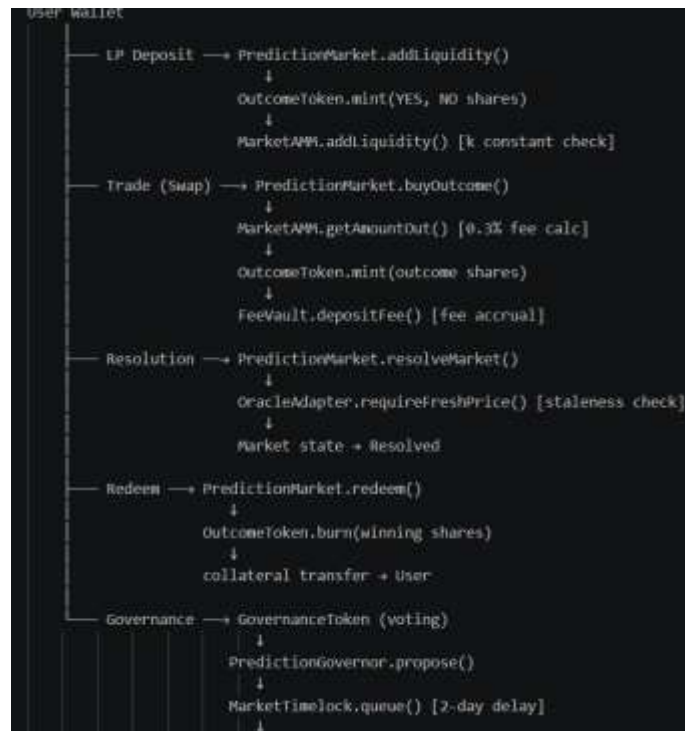
Architecture context diagram



Architecture context diagram



Architecture contract relation work flow and critical work flow



Storage layout & data model

```
// UPGRADEABLE CONTRACT – Layout MUST be preserved across versions

// =====
// STORAGE SLOTS (in order, no gaps!)
// =====

// From Initializable:
// [0x0] uint8 _initialized

// From AccessControlUpgradeable:
// [0x1-0x64] mapping _roles (64 slots for role data)

// From PausableUpgradeable:
// [0x65] bool _paused

// From ReentrancyGuard:
// [0x66] uint256 _status (2 = NOT_ENTERED, 1 = ENTERED)

// PredictionMarket Custom Storage:
slot: [0x100] address collateral // ERC20 token (USDC)
slot: [0x101] address outcomeToken // ERC1155 outcomes
slot: [0x102] address feeVault // ERC4626 fee vault
slot: [0x103] address oracle // OracleAdapter

slot: [0x104] uint256 marketCount // Next market ID

// mappings (use internal slot derivation)
slot: [0x105] mapping(uint256 => Market) markets
struct Market {
    uint256 id;
    string question; // packed in slot [0x106 + offset]
    address oracleFeed;
    int256 resolutionPrice;
    uint256 resolutionTime;
```

```
// NON-UPGRADEABLE – No storage versioning concerns

slot: [0x0] address _owner // AccessControl._owner
slot: [0x1] mapping(bytes32 => RoleData) _roles // AccessControl roles
slot: [0x2] bool _paused // Pausable._paused

// ERC1155 internal:
slot: [0x3] mapping(uint256 => mapping(address => uint256)) _balances
| | | | [tokenId][account] = amount

slot: [0x4] mapping(address => mapping(address => bool)) _operatorApprovals
| | | | [account][operator] = approved

// Outcome-specific:
slot: [0x5] mapping(uint256 => bool) marketResolved
| | | | [marketId] → resolved flag
...

**Token ID Encoding:**
...

tokenId = marketId * 2 → YES outcome
tokenId = marketId * 2 + 1 → NO outcome

Example:
Market #5:
YES token = 5 * 2 = 10
NO token = 5 * 2 + 1 = 11
...
```

Storage layout & data model

```
### 4. **GovernanceToken Storage Layout**
// UPGRADEABLE VIA VUPUpgradeable

// From Initializable:
slot: [0x0] uint8 _initialized

// From AccessControlUpgradeable / OwnableUpgradeable:
slot: [0x1] address _owner

// From ERC20Upgradeable:
slot: [0x2] mapping(address => uint256) _balances
slot: [0x3] mapping(address => mapping(address => uint256)) _allowances
slot: [0x4] uint256 _totalSupply

// From ERC20VotesUpgradeable:
slot: [0x5] mapping(address => Checkpoint[]) _checkpoints
// | | | | | Checkpoint = {blockNumber, votes}
slot: [0x6] mapping(address => uint256) _nonces

// From ERC20PermitUpgradeable:
slot: [0x7] mapping(address => uint256) _nonces_permit

// Custom:
MAX_SUPPLY = 100_000_000e18 // constant, not in storage

// V1-V2 Upgrade Safe: Can add after [0x8]
...

### 5. **MarketAMM (Library - No Storage)**

```solidity
// LIBRARY (embedded in PredictionMarket)
// No independent storage; used as Pool struct within Market

struct Pool {
```

## ### 5. \*\*MarketAMM (Library - No Storage)\*\*

```
```solidity
// LIBRARY (embedded in PredictionMarket)
// No independent storage; used as Pool struct within Market

struct Pool {
    uint256 reserveYes;    // Balance of YES outcome tokens
    uint256 reserveNo;    // Balance of NO outcome tokens
    uint256 totalShares;  // Total LP shares minted
}

// Constant Product:
// reserveYes × reserveNo ≥ k_before (k = product before operation)
// Fees: 0.3% (multiplier 997/1000)
```
```

# ADR

## ### ADR-003: UUPS Proxy Pattern for Core Contracts

### \*\*Context:\*\*

- Markets must be upgradeable to fix bugs or add features.
- Need deterministic upgrade authority and safe storage layout.

### \*\*Options Considered:\*\*

1. **TransparentProxy** – admin in separate slot, simpler but more gas
2. **UUPS** – upgradeable in implementation, compact, requires `_authorizeUpgrade`
3. **No upgrade** – immutable; safer but inflexible

### \*\*Decision:\*\*

**UUPS for PredictionMarket & GovernanceToken**

### \*\*Consequences:\*\*

- Compact proxy (no admin slot); cheaper deployments
- Explicit `_authorizeUpgrade()` control (gated to `UPGRADER_ROLE`)
- Implementation must include `UUPSUpgradeable`; storage layout must be preserved
- Storage collision risk if V2 adds slots incorrectly (mitigated by strict slot ordering)

### \*\*Storage Upgrade Safety (V1-V2):\*\*

1. Document final slot count in V1: `[0x100] ... [0x106]`
2. V2 appends new storage after `[0x107]` **never** inserting in gaps
3. Run foundry slot layout checks: `forge inspect --storage-layout`

**Reference:** `[src/core/PredictionMarket.sol][src/core/GovernanceToken.sol]` Lines 4, 24

---

## ### ADR-002: ERC-1155 for Outcome Tokens

### \*\*Context:\*\*

- Each market has 2 outcomes (YES/NO). Need efficient token representation.
- ERC-20 per outcome would require 2 separate contracts per market (gas-intensive).

### \*\*Options Considered:\*\*

1. **ERC-20 per outcome** – separate token per outcome (inflexible, expensive)
2. **ERC-1155 with tokenId encoding** – single contract, 2 outcomes per market ID
3. **Custom ERC-20 factory** – new contract per market

### \*\*Decision:\*\*

**ERC-1155 with deterministic tokenId encoding**

- `tokenId = marketId * 2 + YES outcome`
- `tokenId = marketId * 2 + 1 + NO outcome`

### \*\*Consequences:\*\*

- Single contract for all market outcomes
- Batch operations via `ERC1155::safeBatchTransferFrom()`
- Standard interface; compatible with aggregators
- TokenID collision if `marketId` overflows (unlikely; `marketCount` capped by `uint256`)
- Some wallets don't fully support ERC-1155 (mitigation: UI hints)

**Reference:** `[src/tokens/OutcomeToken.sol][src/tokens/OutcomeToken.sol]`

## ### ADR-001: CPMM AMM Instead of Order Book

### \*\*Context:\*\*

- Binary prediction markets need deep, continuous liquidity.
- Order books require active market makers; CPMM is passive & capital-efficient.

### \*\*Options Considered:\*\*

1. **Order Book (e.g., Uniswap V4 Hooks)** – explicit maker/taker, complex
2. **CPMM ( $x \cdot y = k$ )** – simple, proven, passive liquidity
3. **Batch Auctions** – complex settlement, latency



# ADR

```
ADR-006: ERC-4626 Fee Vault for Fee Accumulation

Context:
- Market fees (0.3% per swap) accumulate in the protocol.
- Need to incentivize LP deposits & share yield.

Options Considered:
1. Simple token vault - track fees as uint256; no yield
2. ERC-4626 vault - standard interface; compatible with yield strategies
3. Separate fee distributor - complex; multi-step redemptions

Decision:
ERC-4626 Fee Vault (pvSHARE token)

Consequences:
- Standard interface; easy integration with BeFi (Yearn, etc.)
- Shares represent ownership; transparent accounting
- No active yield in V1 (vault is static storage of USDC)
- V2 can delegate to yield strategy (e.g., Aave) without storage changes
- Future upgrade: FeeVault can deploy accumulated fees to earn yield

Fee Distribution Model:
...
User swaps 200 USDC:
 + 0.6 USDC fee (3/1000) + FeeVault
 + FeeVault mints 0.6 pvSHARE shares to LPs (1:1 ratio to assets initially)

Over time:
 + If FeeVault earns 10% yield (via Aave, etc.):
 pvSHARE becomes worth 1.1 USDC per share
 + Existing pvSHARE holders gain value without redemption
...

Reference: src/tokens/FeeVault.sol
```

```
ADR-005: 2-Day Timelock for All DAO Actions

Context:
- DAO can change roles, pause markets, upgrade code.
- Need delay to detect attacks & allow community exit.

Options Considered:
1. No Timelock - instant execution; fast but risky
2. 1-day Timelock - balance; but insufficient for large community
3. 2-day Timelock - standard DAO delay; allows governance + withdrawal
4. 7-day Timelock - very conservative; slows legitimate updates

Decision:
2-day (48-hour) Timelock via MarketTimelock

Consequences:
- Sufficient for community to detect & respond to malicious proposals
- Users have time to exit if upgrade is suspicious
- Slows legitimate governance (9 days total: 1d voting delay + 1w voting + 2d timelock)
- Emergency pause bypasses Timelock (trade-off: trust PAUSER role)

Timeline Example:
...
Block 0: propose() [voting delay starts]
Block 7200: Voting begins (~1 day later @ 12s/block)
Block 57600: Voting ends (~1 week voting period)
| | | queue() [in Timelock]

Day 9: execute() [2 days after queue, now +9d from propose]
...

Reference: src/governance/MarketTimelock.sol
```

```
ADR-004: Chainlink Oracle with 1-Hour Staleness Check

Context:
- Markets need external price feeds to resolve outcomes fairly.
- Chainlink is proven but can be delayed (network congestion, price stability).

Options Considered:
1. Chainlink only - simple but doesn't handle feed outages
2. Chainlink + Fallback (e.g., Uniswap TWAP) - redundancy but complexity
3. Manual oracle (multisig) - flexible but centralized, requires trust
4. Chainlink + 1h staleness check - best balance

Decision:
Chainlink Aggregator with 'requireFreshPrice(message=3600)'

Consequences:
- Industry-standard, decentralized oracle
- Automatic staleness detection (1 hour)
- If price feed is stale >1h, 'resolveMarket()' reverts; requires manual intervention
- Resolver rule must be DAO (not TDA) to prevent censorship

Manual Override Scenario:
...
If Chainlink feed stale for >1h:
 1. DAO votes to trust manual price via governance
 2. RESOLVER_ROLE calls resolveMarket(manualPrice)
 3. Timelock delays 2 days before execution
```



# ADR

## ### ADR-008: Role-Based Access Control (not Ownership)

### \*\*Context:\*\*

- Multiple entities need different permissions (creator, resolver, upgrader, pauser).
- Single 'owner' is too coarse-grained & risky.

### \*\*Options Considered:\*\*

1. **\*\*Single owner\*\*** - simple but all-or-nothing permissions
2. **\*\*Role-based AccessControl\*\*** - granular, hierarchical
3. **\*\*Multisig for each role\*\*** - maximum decentralization but expensive

### \*\*Decision:\*\*

**\*\*OpenZeppelin AccessControl with 5 roles\*\***

### Roles:

- 'DEFAULT\_ADMIN\_ROLE' - admin of admins; grant/revoke other roles
- 'MARKET\_CREATOR\_ROLE' - create new markets
- 'RESOLVER\_ROLE' - resolve market outcomes
- 'PAUSER\_ROLE' - pause/unpause (emergency)
- 'UPGRADER\_ROLE' - upgrade implementation (via UUPS)

### \*\*Consequences:\*\*

- Granular permissions; different teams can manage different functions
- Standard interface; compatible with governance tooling
- All roles initially held by DAO multisig (not auto-decentralized)
- Role revocation requires active governance (not automatic)

### \*\*Governance Transition Path:\*\*

...

V1 (Launch): All roles + DAO Multisig

↓

V2 (Mature): DEFAULT\_ADMIN\_ROLE + Timelock (DAO governs via voting)

MARKET\_CREATOR\_ROLE + Decentralized (anyone can propose markets)

RESOLVER\_ROLE + Chainlink oracle (automated)

...

## ### ADR-009: Reentrancy Guards on State-Changing Functions

### \*\*Context:\*\*

- Market swaps transfer collateral before updating reserves (C-E-I pattern).
- Need to prevent reentrancy attacks via callbacks (ERC777, etc.).

### \*\*Options Considered:\*\*

1. **\*\*No guard\*\*** - risky; callbacks mid-transaction can double-spend
2. **\*\*Checks-Effects-Interactions\*\*** - order matters; still vulnerable to some attacks
3. **\*\*Reentrancy Guard\*\*** - simple mutex; prevents all reentrancy (legitimate & malicious)
4. **\*\*State versioning\*\*** - complex; not worth it for binary markets

### \*\*Decision:\*\*

**\*\*ReentrancyGuard on buyOutcome, addLiquidity, removeLiquidity, redeem\*\***

### \*\*Consequences:\*\*

- Simple, proven protection (OpenZeppelin)
- Minimal gas overhead (~200 gas per guard check)
- Blocks legitimate reentrancy (e.g., flash loans) - acceptable for binary markets
- Not needed on read-only functions (view/pure)

### \*\*Pattern:\*\*

```solidity

```
function buyOutcome(...) external nonReentrant {
    require(status != ENTERED); // Guard check
    _status = ENTERED;
```

```
    // Transfer collateral IN
    collateral.transferFrom(user, this, amount);
```

```
    // Update state
    reserves.update, fees accrue
```

```
    // Transfer tokens OUT
    outcomeToken.safeTransferFrom(...)
```

ADR-007: CREATE2 Deterministic Deployment via MarketFactory

Context:

- For cross-chain compatibility (e.g., Arbitrum + Optimism), need deterministic addresses.
- CREATE (non-deterministic) requires tracking deployments off-chain.

Options Considered:

1. ****CREATE only**** - simple but non-deterministic; breaks cross-chain UX
2. ****CREATE2 only**** - deterministic but requires salt coordination
3. ****CREATE2 with salt mapping**** - hybrid, flexible

Decision:

****CREATE2 deployments with salt + saltToMarket registry****

Consequences:

- Deterministic market addresses across all chains (same salt + same address)
- Cross-chain liquidity pools (e.g., bridge YES tokens from Arbitrum to Optimism)
- Salt collision risk if not unique (mitigation: hash(chainId, marketId, nonce))
- Factory must be owned by DAO (not EOA) to prevent unauthorized deployments

Example:

...

Arbitrum salt: keccak256("ETH058k", chainId=42161) → 0xabcd...

Optimism salt: keccak256("ETH058k", chainId=10) → 0xabcd... (same input, different chain)

Both deploy to SAME ADDRESS on their respective chains

+ Arbitrum market: 0x1234...

+ Optimism market: 0x1234... (same address!)

+ Cross-chain bridges can route correctly

ADR

```
## Design Decisions Log (ADRs)
### ADR-010: Market State Machine (Open → PendingResolution → Resolved)

**Context:**
- Markets have lifecycle: creation → trading → resolution → redemption.
- Need clear state transitions to prevent invalid operations (e.g., trade in Resolved market).

**Options Considered:**
1. **Boolean flags** – `isOpen`, `isResolved` – incomplete state representation
2. **State machine (enum)** – explicit states with valid transitions
3. **Time-based (no explicit state)** – infer from timestamps; fragile

**Decision:**
**Enum-based state machine with validation**

States:
```solidity
enum MarketState {
 Open, // 0: Trading enabled, can add/remove liquidity
 PendingResolution, // 1: No trading; waiting for oracle price
 Resolved, // 2: Outcome finalized; can redeem
 Disputed, // 3: Resolution challenged; waiting for DAO vote
 Cancelled // 4: Market voided (e.g., bad oracle data)
}
```

Transitions:
---
Open → PendingResolution (resolveMarket called, awaiting price)
    → Resolved (price obtained, outcome set)

Resolved → Disputed (someone calls disputeMarket within disputeWindow)

Resolved / Disputed → Cancelled (DAO vote to refund)
```

Summary table and references

Summary Table: Key Decisions

| ADR | Decision | Key Trade-off | Mitigation |
|-----|--------------------|------------------------|-------------------------------|
| 001 | CPMM AMM | Impermanent loss | 0.3% fees to LPs |
| 002 | ERC-1155 outcomes | Low wallet support | UI wallet prompts |
| 003 | UUPS upgrades | Storage collision risk | Slot layout docs + tooling |
| 004 | Chainlink oracle | Feed staleness | Manual resolution override |
| 005 | 2-day Timelock | Slow governance | Necessary for DAO safety |
| 006 | ERC-4626 fee vault | No yield in V1 | Upgrade path prepared |
| 007 | CREATE2 factory | Salt coordination | Registry + hash scheme |
| 008 | Role-based access | Role revocation lag | Governance incentives |
| 009 | Reentrancy guards | Blocks flash loans | Acceptable for binary markets |
| 010 | State machine | Enum storage | Only 1 byte overhead |

References

- [PredictionMarket.sol](src/core/PredictionMarket.sol) – Main market contract
- [MarketAMM.sol](src/core/MarketAMM.sol) – CPMM swap logic (library)
- [OutcomeToken.sol](src/tokens/OutcomeToken.sol) – ERC-1155 token encoding
- [GovernanceToken.sol](src/tokens/GovernanceToken.sol) – Voting token (UUPS)
- [FeeVault.sol](src/tokens/FeeVault.sol) – ERC-4626 vault
- [OracleAdapter.sol](src/oracles/OracleAdapter.sol) – Chainlink wrapper
- [PredictionGovernor.sol](src/governance/PredictionGovernor.sol) – Governance contract
- [MarketTimelock.sol](src/governance/MarketTimelock.sol) – Timelock controller
- [MarketFactory.sol](src/factories/MarketFactory.sol) – Deployment factory
- [Deploy.s.sol](deploy/Deploy.s.sol) – Deployment script

Trust assumptions

1. **Access Control Hierarchy**

| Role | Holder | Powers | Revocation |
|---------------------|--------------|---|---------------|
| DEFAULT_ADMIN_ROLE | DAO Multisig | Grant/revoke all roles, pause, upgrade | Timelock (2d) |
| MARKET_CREATOR_ROLE | DAO Multisig | Create new markets | Timelock (2d) |
| RESOLVER_ROLE | DAO Multisig | Resolve markets (finalize outcomes) | Timelock (2d) |
| PAUSER_ROLE | DAO Multisig | Pause market (emergency) | Timelock (2d) |
| UPGRADER_ROLE | DAO Multisig | Upgrade PredictionMarket implementation | Timelock (2d) |

2. **Oracle Trust Model**

| Component | Trust Assumption | Risk | Mitigation |
|----------------------|-----------------------------------|---|--|
| Chainlink Aggregator | Price feed is accurate & recent | Stale/incorrect price + wrong outcome | requireFreshPrice(maxAge=3600) checks. Manual override via Resolver. |
| OracleAdapter | Adapter correctly wraps Chainlink | Wrapper bug → market resolution fails | Simple, audited wrapper. No computation. |
| Resolver Role | Trusted operator calls resolve | Collusion/censorship → markets unresolved | DAO governance + Timelock on role changes. |

3. **Multisig/Admin Compromise**

If DAO Multisig Compromised:

| Action | Impact | Timelock Mitigates | Monitoring |
|--------------------------------|---|---------------------------|---|
| grantRole(UPGRADER, attacker) | Attacker can deploy malicious PredictionMarket V2 | ✓ 2-day delay to revoke | Monitor role grant events; community vote to revoke |
| pause() / unpause() | Market frozen / unfrozen | X Immediate (PAUSER_ROLE) | Community monitoring; pause only in emergencies |
| removeRole(RESOLVER) | Markets can't resolve | X Immediate | Restore role via governance or manual multisig |
| setImplementation() in Factory | New markets deploy to malicious code | X No Timelock on Factory | Factory ownership should be DAO/Timelock, not EOA |

АКТИВАЦИЯ WINDOWS

Чтобы активировать Windows, перейди

Architecture

- - ****CPMM AMM****: $x*y=k$ with 0.3% fee, Yul-optimized swap calculation
- - ****ERC-1155 Outcome Tokens****: YES/NO shares per market ($\text{tokenId} = \text{marketId} * 2 / \text{marketId} * 2 + 1$)

```
function buyOutcome(
    uint256 marketId,
    bool    isYes,
    uint256 amountIn,
    uint256 minOut
) external nonReentrant whenNotPaused {
    if (amountIn == 0) revert ZeroAmount();
    Market storage m = _requireOpen(marketId);

    uint256 fee = (amountIn * FEE_PERCENT) / 1000;
    uint256 amountInAfterFee = amountIn - fee;

    uint256 reserveIn  = isYes ? m.pool.reserveNo  : m.pool.reserveYes;
    uint256 reserveOut = isYes ? m.pool.reserveYes : m.pool.reserveNo;

    uint256 amountOut = MarketAMM.getAmountOut(amountInAfterFee, reserveIn, reserveOut);
    if (amountOut < minOut) revert SlippageTooHigh();

    // Effects
    if (isYes) {
        m.pool.reserveYes -= amountOut;
        m.pool.reserveNo  += amountInAfterFee;
    } else {
        m.pool.reserveNo  -= amountOut;
        m.pool.reserveYes += amountInAfterFee;
    }
    m.totalCollateral += amountIn;

    collateral.safeTransferFrom(msg.sender, address(this), amountIn);

    collateral.approve(address(feeVault), fee);
    feeVault.depositFee(fee);

    uint256 tokenId = isYes ? _yesId(marketId) : _noId(marketId);
    outcomeToken.mint(msg.sender, tokenId, amountOut, "");
}
```

Architecture

- **ERC-4626 Fee Vault**
- LP fee accumulation with yield sharing

```
FeeVault is ERC4626, Ownable {
    using SafeERC20 for IERC20;

    address public market;

    error OnlyMarket();

    event FeeDeposited(uint256 amount);

    constructor(
        IERC20 asset_,
        address owner_
    )
        ERC4626(asset_)
        ERC20("PredictVaultShares", "pvSHARE")
        Ownable(owner_)
    {}

    function setMarket(address market_) external onlyOwner {
        market = market_;
    }

    function depositFee(uint256 amount) external {
        if (msg.sender != market) revert OnlyMarket();
        IERC20(asset()).safeTransferFrom(msg.sender, address(this), amount);
        emit FeeDeposited(amount);
    }

    function totalAssets() public view override returns (uint256) {
        return IERC20(asset()).balanceOf(address(this));
    }
}
```


Chainlink Oracle

```
function getPrice(address feed) external view returns (int256 price, uint256 updatedAt);
function requireFreshPrice(address feed, uint256 maxAge) external view returns (int256);
}

contract OracleAdapter is IOracleAdapter {
    error StalePrice(address feed, uint256 updatedAt, uint256 maxAge);
    error InvalidPrice(address feed, int256 price);

    /// @notice Возвращает цену и время обновления
    function getPrice(address feed)
        public view returns (int256 price, uint256 updatedAt)
    {
        (
            uint80 roundId,
            int256 answer,
            ,
            uint256 updateTime,
            uint80 answeredInRound
        ) = AggregatorV3Interface(feed).latestRoundData();

        require(answeredInRound >= roundId, "Stale round");
        require(answer > 0, "Negative price");

        return (answer, updateTime);
    }

    /// @notice Reverts если цена старше maxAge секунд
    function requireFreshPrice(
        address feed,
        uint256 maxAge
    ) external view returns (int256) {
        (int256 price, uint256 updatedAt) = getPrice(feed);
        if (block.timestamp - updatedAt > maxAge) {
            revert StalePrice(feed, updatedAt, maxAge);
        }
    }
}
```

```
import "@chainlink/contracts/src/v0.8/shared/interfaces/AggregatorV3Interface.sol";

contract MockAggregator is AggregatorV3Interface {
    int256 private _answer;
    uint256 private _updatedAt;
    uint8 private _decimals;

    constructor(int256 answer_, uint8 decimals_) {
        _answer = answer_;
        _updatedAt = block.timestamp;
        _decimals = decimals_;
    }

    function setAnswer(int256 answer_) external {
        _answer = answer_;
        _updatedAt = block.timestamp;
    }

    function setUpdatedAt(uint256 t) external { _updatedAt = t; }

    function decimals() external view returns (uint8) { return _decimals; }
    function description() external pure returns (string memory) { return "Mock"; }
    function version() external pure returns (uint256) { return 1; }

    function latestRoundData() external view returns (
        uint80 roundId, int256 answer, uint256 startedAt,
        uint256 updatedAt, uint80 answeredInRound
    ) {
        return (1, _answer, _updatedAt, _updatedAt, 1);
    }

    function getRoundData(uint80) external view returns (
        uint80, int256, uint256, uint256, uint80
    ) {
        return (1, _answer, _updatedAt, _updatedAt, 1);
    }
}
```

UUPS Upgradable (PredictionMarket and GovernanceToken with V1→V2 upgrade path)

```
contract PredictionGovernor is
    Governor,
    GovernorSettings,
    GovernorCountingSimple,
    GovernorVotes,
    GovernorVotesQuorumFraction,
    GovernorTimelockControl
{
    constructor(
        IVotes token_,
        TimelockController timelock_
    )
        Governor("PredictionGovernor")
        GovernorSettings(
            7200, // voting delay ~1 day @ 12s/block
            50400, // voting period ~1 week @ 12s/block
            1e18 // proposal threshold
        )
        GovernorVotes(token_)
        GovernorVotesQuorumFraction(4) // 4% quorum
        GovernorTimelockControl(timelock_)
    {}

    // Required overrides
    function votingDelay() public view override(Governor, GovernorSettings) returns (uint256) {
        return super.votingDelay();
    }

    function votingPeriod() public view override(Governor, GovernorSettings) returns (uint256) {
        return super.votingPeriod();
    }

    function quorum(uint256 blockNumber)
        public view override(Governor, GovernorVotesQuorumFraction) returns (uint256)
    {
        return super.quorum(blockNumber);
    }
}
```

Актив
Чтобы
"Пара

```
// V1 -> V2 upgrade path: добавить vesting module, override _authorizeUpgrade
function _authorizeUpgrade(address newImpl) internal override onlyOwner {}

function _update(
    address from,
    address to,
    uint256 amount
) internal override(ERC20Upgradeable, ERC20VotesUpgradeable) {
    super._update(from, to, amount);
}

function nonces(
    address owner
)
    public
    view
    override(ERC20PermitUpgradeable, NoncesUpgradeable)
    returns (uint256)
{
    return super.nonces(owner);
}
```

DAO Governance

```
contract PredictionGovernor is
    Governor,
    GovernorSettings,
    GovernorCountingSimple,
    GovernorVotes,
    GovernorVotesQuorumFraction,
    GovernorTimelockControl
{
    constructor(
        IVotes token_,
        TimelockController timelock_
    )
    {
        Governor("PredictionGovernor")
        GovernorSettings(
            7200,          // voting delay ~1 day @ 12s/block
            50400,         // voting period ~1 week @ 12s/block
            1e18           // proposal threshold
        )

        GovernorVotes(token_)
        GovernorVotesQuorumFraction(4) // 4% quorum
        GovernorTimelockControl(timelock_)
    }

    // Required overrides
    function votingDelay() public view override(Governor, GovernorSettings) returns (uint256) {
        return super.votingDelay();
    }

    function votingPeriod() public view override(Governor, GovernorSettings) returns (uint256) {
        return super.votingPeriod();
    }

    function quorum(uint256 blockNumber)
    public view override(Governor, GovernorVotesQuorumFraction) returns (uint256)
    {
        return super.quorum(blockNumber);
    }
}
```

```

{
    return super.proposalThreshold();
}

function state(uint256 proposalId)
    public view override(Governor, GovernorTimelockControl) returns (ProposalState)
{
    return super.state(proposalId);
}

function proposalNeedsQueuing(uint256 proposalId)
    public view override(Governor, GovernorTimelockControl) returns (bool)
{
    return super.proposalNeedsQueuing(proposalId);
}

function _queueOperations(
    uint256 proposalId,
    address[] memory targets,
    uint256[] memory values,
    bytes[] memory calldatas,
    bytes32 descriptionHash
) internal override(Governor, GovernorTimelockControl) returns (uint48) {
    return super._queueOperations(proposalId, targets, values, calldatas, descriptionHash);
}

function _executeOperations(
    uint256 proposalId,
    address[] memory targets,
    uint256[] memory values,
    bytes[] memory calldatas,
    bytes32 descriptionHash
) internal override(Governor, GovernorTimelockControl) {
    super._executeOperations(proposalId, targets, values, calldatas, descriptionHash);
}
}
```

```
function _cancel(
    address[] memory targets,
    uint256[] memory values,
    bytes[] memory calldatas,
    bytes32 descriptionHash
) internal override(Governor, GovernorTimelockControl) returns (uint256) {
    return super._cancel(targets, values, calldatas, descriptionHash);
}

function _executor()
    internal view override(Governor, GovernorTimelockControl) returns (address)
{
    return super._executor();
}
}
```

DAO Governance and Deploy script

```
function run() external {
    uint256 deployerKey = vm.envUint("PRIVATE_KEY");
    address deployer = vm.addr(deployerKey);
    vm.startBroadcast(deployerKey);
    MockUSDC usdc = new MockUSDC(deployer);
    usdc.mint(deployer, 1_000_000e6);
    address collateral = address(usdc);
    console.log("MockUSDC: ", collateral);

    GovernanceToken govImpl = new GovernanceToken();
    ERC1967Proxy govProxy = new ERC1967Proxy(
        address(govImpl),
        abi.encodeCall(GovernanceToken.initialize, (deployer, deployer))
    );
    GovernanceToken govToken = GovernanceToken(address(govProxy));

    address[] memory proposers = new address[](1);
    address[] memory executors = new address[](1);
    proposers[0] = address(0);
    executors[0] = address(0);

    MarketTimelock timelock = new MarketTimelock(proposers, executors, deployer);

    PredictionGovernor governor = new PredictionGovernor(
        1Votes(address(govToken)),
        TimelockController(payable(address(timelock)))
    );

    timelock.grantRole(timelock.PROPOSER_ROLE(), address(governor));
    timelock.grantRole(timelock.CANCELLER_ROLE(), address(governor));

    OutcomeToken outcomeToken = new OutcomeToken(deployer);

    Endpoint endpoint = new Endpoint(TECMA(collateral), deployer);
```

```
contract MarketTimelock is TimelockController {
    constructor(
        address[] memory proposers,
        address[] memory executors,
        address admin
    )
        TimelockController(
            2 days,
            proposers,
            executors,
            admin
        )
    {}
}
```

```
        abi.encodeCall(PredictionMarket.initialize, (
            deployer,
            collateral,
            address(outcomeToken),
            address(feeVault),
            address(oracle)
        ));
    PredictionMarket market = PredictionMarket(address(marketProxy));

    outcomeToken.grantRole(outcomeToken.MINTER_ROLE(), address(market));
    feeVault.setMarket(address(market));

    MarketFactory factory = new MarketFactory(address(marketImpl), address(timelock));

    vm.stopBroadcast();

    console.log("=== DEPLOYED ADDRESSES ===");
    console.log("govToken: ", address(govToken));
    console.log("timelock: ", address(timelock));
    console.log("governor: ", address(governor));
    console.log("outcomeToken: ", address(outcomeToken));
    console.log("feeVault: ", address(feeVault));
    console.log("oracle: ", address(oracle));
    console.log("Market: ", address(market));
    console.log("Factory: ", address(factory));
}
```